NEO-PAC

An Autonomous Real-Time Path-Planning and Evasion System

Semester Project Proposal — BAI-4A

Group Members

Sr.	Name	Roll Number	Section
1	Abdul Hanan	24i-0087	BAI-4A
2	Asjad Kamal	24i-0046	BAI-4A
3	Abdul Azeem	24i-2013	BAI-4A
4	Abdul Rauf	24i-0060	BAI-4A
5	Fauzan Tahir	24i-0042	BAI-4A

Category

Mobility. Neo-Pac features an autonomous mobile robot (Pac-Man) built on a differential drive (2-wheeled) chassis. Movement is fully autonomous — the robot calculates its own trajectory in real time based on overhead camera data and AI pathfinding. No manual remote control is required during operation.

Description

Neo-Pac is a new project idea that brings classic arcade logic into the physical world. The goal is to build a tabletop maze where a robot representing Pac-Man must navigate and survive while being hunted by a Ghost robot controlled either autonomously or manually.

The Story: Static environments make pathfinding trivial. Neo-Pac introduces a dynamic cat-and-mouse challenge. An overhead camera continuously captures the full maze. A central PC processes this feed using OpenCV to detect color-coded markers on each robot and converts the frame into a 2D grid map. The AI then predicts the Ghost's next moves and calculates a safe path for Pac-Man using BFS/Dijkstra. The path is packaged as movement commands and sent wirelessly via UDP to the Pac-Man robot, which executes them using PID-controlled motors. Pac-Man also collects pellets placed across the maze grid, adding a game objective on top of the evasion task.

Main Hardware Challenge:

Maintaining low-latency wireless communication between the central PC and the robot so that movement commands arrive fast enough to avoid the Ghost in real time.

Main Software Challenge:

Implementing a dynamic pathfinding algorithm that fully recalculates the safe path within milliseconds every time the Ghost moves, while also factoring in predicted future Ghost positions.

Use Case

Neo-Pac is an educational and research-oriented platform demonstrating how AI can handle dynamic obstacle avoidance in a constrained grid environment. It is relevant to robotics research, AI education, and autonomous systems demonstrations.

User Interaction Table:

User Action	Product Reaction
User places Ghost in the maze	Pac-Man detects the threat and instantly recalculates a safe path away from the Ghost.
User blocks a path with a physical object	Perception module updates the grid map and Pac-Man reroutes around the new obstacle.
User increases Ghost speed	Path planner shifts to a more conservative, longer-range strategy to stay ahead.
Pac-Man reaches a pellet cell	Pellet is marked as collected, score updates on the live display, and Pac-Man continues pathing.
Ghost catches Pac-Man (overlap on grid)	System logs a collision event, game resets or pauses, and result is shown on screen.

Modules

1. Locomotion Module

Handles all physical movement of the Pac-Man robot. The module receives directional commands (North, South, East, West) from the Command & Control module and translates them into motor PWM signals. PID control is used to ensure smooth, accurate turns and consistent speed between grid cells. The module also handles stopping precisely at cell centers to maintain grid alignment.

Algorithm/Technique: PID Control, PWM Motor Control

2. Perception Module (Computer Vision)

Processes the live overhead camera feed to understand the state of the maze. OpenCV is used to detect color-coded markers on Pac-Man and the Ghost robot, extract their pixel coordinates, and map those coordinates onto a logical 2D grid. The module also detects physical obstacles placed in the maze and marks them as blocked cells. This grid map is continuously updated and fed to the AI modules.To improve robustness under varying lighting conditions, a pretrained object detection model (YOLOv8) will also be integrated into the perception pipeline. The model processes frames from the overhead camera and detects the Pac-Man and Ghost robots using bounding box localization. The center of each detected bounding box is mapped to the corresponding grid cell in the maze. This allows reliable robot tracking even if color detection fails or the environment lighting changes.

Algorithm/Technique: OpenCV Color Detection, Contour Detection, Coordinate-to-Grid Mapping, Pretrained YOLOv8 Object Detection Model

3. Command & Control Module

Acts as the communication bridge between the PC (AI Brain) and the Pac-Man robot. It takes the planned path from the Safe Path Planner, breaks it down into step-by-step movement commands, packages them into UDP packets, and transmits them over Wi-Fi to the ESP32 on the robot. It also receives acknowledgment signals back from the robot to confirm execution before sending the next command.

Algorithm/Technique: UDP Socket Communication, Command Queuing

4. Safe Path Planner (AI Module 1 — Custom)

The core AI module of the project. It takes the current grid map from the Perception Module and the Ghost's predicted future positions from the Ghost Movement Predictor, then runs a modified BFS or Dijkstra's algorithm to find the shortest path that avoids both current and predicted danger zones. Ghost-adjacent cells are assigned high traversal costs to steer Pac-Man away from them. The path is recalculated every time a new frame is processed.

Algorithm/Technique: BFS / Dijkstra's Algorithm with dynamic cost weighting

5. Ghost Movement Predictor (AI Module 2 — Custom)

A heuristic-based prediction module that analyzes the Ghost's current position, velocity, and direction to forecast its next 2-3 moves. It uses Manhattan distance heuristics and rule-based directional weighting to estimate where the Ghost is heading. These predictions are passed to the Safe Path Planner so it can avoid zones that are about to become dangerous, not just zones that are currently dangerous.

Algorithm/Technique: Manhattan Distance Heuristic, Rule-Based Directional Weighting

6. Object Detector (AI Module 3 — Pre-Trained)

To improve robustness in real-world conditions, a pretrained computer vision model will also be integrated into the perception pipeline. A pretrained **YOLOv8 object detection model** will process frames captured by the overhead camera and detect the Pac-Man and Ghost robots using bounding box localization. The center coordinates of each detected bounding box will then be converted into grid positions within the maze environment. These coordinates are passed to the Safe Path Planner and Ghost Movement Predictor modules for decision-making. Using a pretrained model allows the system to perform reliable robot detection even under varying lighting conditions, partial occlusion, or color detection failures.

Algorithm/Technique:
Pretrained YOLOv8 Object Detection, Bounding Box Localization, Coordinate-to-Grid Mapping

Hardware Components

- Microcontroller: ESP32 (Wi-Fi enabled, handles UDP communication and motor control)
- Chassis: 2WD Robot Kit with DC Gear Motors
- Motor Driver: L298N or L293D
- Overhead Camera: Logitech USB Camera (wide angle, mounted above maze)
- Power: 2x 18650 Li-ion batteries with a 5V voltage regulator

- Maze Structure: Custom-built modular foam/wood board with grid markings

Contribution

Hardware

- Custom modular maze structure with grid-marked floor for robot navigation
- Customized robot chassis optimized for low-friction turns on maze surface

Software

- Custom Python scripts for OpenCV-based robot tracking and grid mapping
- Custom BFS/Dijkstra pathfinding implementation specialized for dynamic grid with moving obstacles
- Ghost movement prediction module using Manhattan distance and directional heuristics
- Real-time GUI for live visualization of Pac-Man position, Ghost position, and planned safe path

Outsourcing

Hardware

- Pre-made gearboxes, wheels, and standard battery holders
- Off-the-shelf motor driver boards (L298N)

Software

- OpenCV library for image processing and contour detection
- NumPy for matrix-based grid calculations
- Python socket library for UDP communication

ROS2 Implementation

ROS2 is not being used in the current plan. However, if it becomes mandatory, the project will be modularized into the following ROS2 nodes without changing any core project requirements:

- vision_node: Processes camera feed and publishes robot coordinates on the /positions topic
- path_planner_node: Subscribes to /positions and publishes the calculated safe path on /planned_path
- predictor_node: Subscribes to /positions and publishes Ghost predictions on /ghost_prediction
- robot_driver_node: Subscribes to /planned_path and translates it into /cmd_vel commands for the robot

Communication

- Wi-Fi (UDP Protocol): Real-time movement command exchange between the PC (AI Brain) and the Pac-Man robot's ESP32. UDP is chosen over TCP for lower latency.

- I2C: Internal communication between the ESP32 and any onboard sensors or secondary controllers on the robot chassis.

Minimum Viable Product (MVP)

The project will be considered complete when the following conditions are met:

1. The Pac-Man robot can autonomously navigate from Point A to Point B on a 4x4 grid.
2. The system detects one Ghost and moves Pac-Man to a safe cell in real time.
3. The live safe path is visualized on a laptop screen via a GUI showing Pac-Man, Ghost, and path.
4. The system includes all 5 modules and handles the main software challenge of dynamic pathfinding.
5. Low-latency Wi-Fi communication is established between the PC and the robot with command acknowledgment.

System Architecture Diagram